

Lab 4 Pie Chart Preprocessing and conditional logic

2026-01-16

Week 4 Preprocessing

Goal

Create a Pie Chart to show the distribution of runs by Time of Day.

Imports

```
library(dplyr) # We will now use dplyr as standard

##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union

file_id <- "1ymbNqfv9s6YGZzN93HFKAhjg0Z5xZXV1"
url <- paste0("https://drive.google.com/uc?id=", file_id)

df <- read.csv(url)
```

Create frequency table for runs

We want to create a dataframe that counts how many days contain runs in the AM and PM. Since we cannot assume there is only one run per day, we also include a third category called Both, for days where runs occurred in both the morning and afternoon/evening.

Using either the steps below or your own method, create a summary table showing the number of days in each category (AM, PM, and Both).

1. Create new columns for datetime, Date, AMPM

We first parse the `start_date` string into a proper date-time column called `datetime` using:

```
as.POSIXct(start_date, format = "%Y-%m-%dT%H:%M:%SZ", tz = "UTC")
```

Once we have `datetime`, we can:

- Use `as.Date(datetime)` to extract the calendar date (year-month-day). This can then be used to group activities by day.
- Use `format(datetime, "%p")` to extract whether the activity occurred in the AM or PM.

In the model answer below, I use these steps as a template to build the required summary table.

```
##           Date AMPM
## 1 2025-07-21   PM
## 2 2025-07-19   AM
## 3 2025-07-16   PM
## 4 2025-07-15   AM
## 5 2025-07-12   AM
## 6 2025-07-08   PM
```

2. Group by

We can pipe `%>%` our data into `group_by(Date)` to group activities by day. This tags each row with a group label based on the `Date` column, but it does not perform any calculations by itself. Calculations are typically performed afterwards using functions such as `summarise()`.

```
## # A tibble: 6 x 2
## # Groups:   Date [6]
##   Date      AMPM
##   <date>    <chr>
## 1 2025-07-21 PM
## 2 2025-07-19 AM
## 3 2025-07-16 PM
## 4 2025-07-15 AM
## 5 2025-07-12 AM
## 6 2025-07-08 PM
```

3. Conditional Logic

We now want to create a daily summary, but we also need to apply a condition. One approach is to count how many unique values of `AMPM` occur on each day. If there are two (i.e., both AM and PM), we label that day as “Both”.

In R, we can apply conditional logic using:

```
ifelse(condition, value_if_true, value_if_false)
```

We use this inside `summarise()` as `summarise()` collapses multiple rows per day into a single row by calculating summary values (i.e., aggregating the data).

Using the example below (or your own method), apply this logic to your dataframe.

```
# Quick example
example <- data.frame(
  Date = as.Date(c("2025-07-19", "2025-07-19", "2025-07-20", "2025-07-21", "2025-07-21")),
  AMPM = c("AM", "PM", "AM", "PM", "PM")
) %>%
  group_by(Date) %>%
  summarise(
    period = ifelse(n_distinct(AMPM) == 2, "Both", first(AMPM)),
    .groups = "drop"
  )

example
```

```
## # A tibble: 3 x 2
##   Date      period
##   <date>    <chr>
## 1 2025-07-19 Both
## 2 2025-07-20 AM
## 3 2025-07-21 PM
```

The actual output you should aim for is below;

```
## # A tibble: 6 x 2
##   Date      period
##   <date>    <chr>
## 1 2011-01-05 AM
## 2 2011-01-10 AM
## 3 2011-01-17 AM
## 4 2011-01-21 AM
## 5 2011-01-27 AM
## 6 2011-02-02 AM
```

4. Provide counts

Provide counts. This can be done in a single operation.

```
## # A tibble: 3 x 2
##   period      n
##   <chr> <int>
## 1 AM      491
## 2 Both       7
## 3 PM      421
```

5. Save to csv